

Java Backend Interview Series

Sub-Chapter 1.6

Collections -- Internals, Algorithms & Best Practices

ArrayList / LinkedList / HashMap / Iterators / Comparators / Streams

Complete Import Statements -- 30+ Q&A -- Working Examples -- Pitfalls -- Exercises

1 Collection Framework Overview

Before Java 2, developers relied on Vector, Hashtable, and raw arrays with no unified API. Java 2 (JDK 1.2) introduced the Collections Framework: a unified hierarchy of interfaces backed by concrete implementations. Understanding this hierarchy is the foundation for every data-structure question in Java interviews.

1.1 The Hierarchy -- Text Diagram

```
java.lang.Iterable<E>

java.util.Collection<E>

java.util.List<E> -- ordered, duplicates OK

ArrayList<E> -- dynamic array, O(1) random access

LinkedList<E> -- doubly-linked, O(1) ends

CopyOnWriteArrayList<E> -- snapshot-on-write, thread-safe

java.util.Set<E> -- no duplicates

HashSet<E> -- backed by HashMap, O(1) ops

LinkedHashSet<E> -- HashSet + insertion order

TreeSet<E> -- Red-Black tree, O(log n)

EnumSet<E> -- bitset-backed, fastest Set

java.util.Queue<E> -- FIFO

ArrayDeque<E> -- resizable-array Deque

PriorityQueue<E> -- min-heap, O(log n) poll

java.util.Deque<E> -- double-ended queue

ArrayDeque<E> -- preferred over Stack/LinkedList

java.util.Map<K,V> -- key-value pairs (NOT Collection)

HashMap<K,V> -- O(1) ops, no order

LinkedHashMap<K,V> -- HashMap + insertion/access order

TreeMap<K,V> -- Red-Black tree, sorted keys

EnumMap<K,V> -- ordinal-array, fastest Map

ConcurrentHashMap<K,V> -- lock-striping, thread-safe
```

Java Collections Framework hierarchy

1.2 Iterator vs ListIterator vs Spliterator

Feature	Iterator	ListIterator	Spliterator
Direction	Forward only	Forward + backward	Forward (tryAdvance)

Feature	Iterator	ListIterator	Spliterator
Index access	No	nextIndex/previousIndex	No
Add elements	No	Yes (add())	No
Set elements	No	Yes (set())	No
Safe remove	Yes (remove())	Yes (remove())	No
Parallel	No	No	Yes (trySplit)
Source	Any Collection	List only	Any Collection/Array
Primary use	Sequential traversal	Bidirectional list scan	Parallel stream pipeline

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.ListIterator;
import java.util.Spliterator;

// Iterator -- forward only, safe remove
List<String> names = new ArrayList<>(List.of("Alice", "Bob", "Carol"));
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String n = it.next();
    if (n.startsWith("B")) it.remove(); // safe structural remove
}

// ListIterator -- bidirectional
ListIterator<String> li = names.listIterator(names.size());
while (li.hasPrevious()) System.out.print(li.previous() + " ");

// Spliterator -- stream internals
Spliterator<String> spl = names.spliterator();
System.out.println("Est. size: " + spl.estimateSize());
spl.forEachRemaining(System.out::println);

```

Iterator / ListIterator / Spliterator usage

2 ArrayList Internals

ArrayList is a resizable-array implementation backed by a plain `Object[]` field named `elementData`. Understanding how it grows, shrinks, and copies is essential for predicting performance in bulk-insert or memory-sensitive scenarios.

2.1 Internal Fields and DEFAULT_CAPACITY

```
import java.util.ArrayList;

// Simplified ArrayList source (java.util.ArrayList, OpenJDK)

public class ArrayList<E> extends AbstractList<E> {

    private static final int DEFAULT_CAPACITY = 10;

    private static final Object[] DEFAULTCAPACITY_EMPTY_ELEMENTDATA = {};

    transient Object[] elementData; // backing array

    private int size; // logical size

    // No-arg constructor -- LAZY: starts with 0-length array
    public ArrayList() {

        this.elementData = DEFAULTCAPACITY_EMPTY_ELEMENTDATA;

    }

    // Capacity constructor
    public ArrayList(int initialCapacity) {

        this.elementData = new Object[initialCapacity];

    }

}
```

ArrayList internal fields (simplified OpenJDK source)

2.2 grow() -- 1.5x Expansion

```
import java.util.ArrayList;
import java.util.Arrays;

// grow() called when size == elementData.length
// New capacity = oldCapacity + (oldCapacity >> 1) -- 1.5x

private Object[] grow(int minCapacity) {

    int oldCapacity = elementData.length;

    if (oldCapacity > 0) {

        int newCapacity = oldCapacity + (oldCapacity >> 1); // 1.5x
        return elementData = Arrays.copyOf(elementData, newCapacity);

    } else {

        // First add: jump to DEFAULT_CAPACITY (10)

    }

}
```

```

return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
}
}

// Capacity growth milestones for new ArrayList():
// add #1 -> capacity 10 (first add jumps to DEFAULT_CAPACITY)
// add #11 -> grow(): 10 + 5 = 15
// add #16 -> grow(): 15 + 7 = 22
// add #23 -> grow(): 22 + 11 = 33

// Pre-size to avoid resizing:
ArrayList<Integer> presized = new ArrayList<>(1_000_000);
// trimToSize() reclaims unused capacity after bulk-fill:
presized.trimToSize();

```

ArrayList grow() -- 1.5x expansion logic

2.3 System.arraycopy Cost and trimToSize

Arrays.copyOf uses System.arraycopy internally -- a native JVM call, faster than a Java loop, but still $O(n)$ for each resize. For n add() calls, total work is $O(n)$ amortized (geometric series), but individual grow() calls spike latency. Use ensureCapacity(n) before bulk inserts to eliminate all intermediate copies.

```

import java.util.ArrayList;
import java.util.List;

// Best practice: pre-size before large inserts
List<String> rows = new ArrayList<>();

rows.ensureCapacity(500_000); // one allocation, no resizing
for (int i = 0; i < 500_000; i++) rows.add("row-" + i);
rows.trimToSize(); // free unused tail capacity

// System.arraycopy cost on remove(int index):
// Shifts all elements right-of-index left by 1
// Cost:  $O(n - \text{index})$  -- removing from index 0 is most expensive
rows.remove(0); // shifts 499,999 elements -- avoid in hot paths

```

ensureCapacity and trimToSize best practices

WARNING: Calling remove(0) on a large ArrayList is $O(n)$. Use ArrayDeque or LinkedList when you frequently remove from both ends.

TIP: Pre-size ArrayList with the expected count: new ArrayList<>(expectedCount) eliminates all intermediate grow() calls.

3 LinkedList Internals

LinkedList is a doubly-linked list that also implements Deque. Each element is wrapped in a Node object with forward and backward references. This gives $O(1)$ add/remove at both ends but $O(n)$ for index-based access, making it unsuitable as a general-purpose List replacement.

3.1 Node Structure and Core Operations

```
import java.util.LinkedList;
import java.util.Deque;

// OpenJDK LinkedList internals (simplified)

public class LinkedList<E> extends AbstractSequentialList<E>
    implements List<E>, Deque<E> {

    private static class Node<E> {
        E item;
        Node<E> next;
        Node<E> prev;
        Node(Node<E> prev, E element, Node<E> next) {
            this.item = element; this.next = next; this.prev = prev;
        }
    }

    transient int size = 0;
    transient Node<E> first; // head
    transient Node<E> last; // tail

    // linkLast -- O(1) append
    void linkLast(E e) {
        final Node<E> l = last;
        final Node<E> newNode = new Node<>(l, e, null);
        last = newNode;
        if (l == null) first = newNode; else l.next = newNode;
        size++; modCount++;
    }
}
```

LinkedList Node structure and linkLast (OpenJDK simplified)

3.2 Complexity and When to Use

Operation	LinkedList	ArrayList	ArrayDeque
add(E) / addLast	$O(1)$	$O(1)$ amortized	$O(1)$ amortized

Operation	LinkedList	ArrayList	ArrayDeque
addFirst	O(1)	O(n) shift	O(1) amortized
get(index)	O(n) walk	O(1)	No index access
remove(index)	O(n) walk + O(1) unlink	O(n) shift	No index access
removeFirst / poll	O(1)	O(n) shift	O(1)
Memory per element	~40 bytes (Node overhead)	~4 bytes (ref in array)	~4 bytes

```
import java.util.Deque;
import java.util.LinkedList;

// LinkedList as a Queue (prefer ArrayDeque in practice)
Deque<String> queue = new LinkedList<>();
queue.offer("first"); // addLast -- O(1)
queue.offer("second");
String head = queue.poll(); // removeFirst -- O(1)

// Walking by index is O(n) -- halved by starting from nearest end:
// if (index < size/2) walk from first; else walk from last
```

LinkedList as Queue

WARNING: Prefer ArrayDeque over LinkedList for both Queue and Stack use-cases. ArrayDeque has better cache locality and no per-element Node allocation overhead.

4 HashMap Internals

HashMap is the most important Map implementation and one of the most-asked internals topics in Java interviews. It stores entries in a `Node[]` table array, uses hash spreading to distribute keys, handles collisions with linked lists or balanced trees, and resizes by doubling capacity when the load factor threshold is exceeded.

4.1 Entry Storage and Hash Spreading

```
import java.util.HashMap;
import java.util.Map;

// Simplified HashMap Node and hash() method (OpenJDK)

static class Node<K,V> implements Map.Entry<K,V> {

    final int hash;

    final K key;

    V value;

    Node<K,V> next; // collision chain

}

// Hash spreading: XOR high 16 bits into low 16 bits
// Reduces collisions when table is small (low bits dominate)
static final int hash(Object key) {

    int h;

    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);

}

// Bucket index: (n-1) & hash (n = table.length, always power of 2)
int index = (n - 1) & hash(key);

static final int DEFAULT_INITIAL_CAPACITY = 16; // power of 2
static final float DEFAULT_LOAD_FACTOR = 0.75f;

// Resize when: size > capacity * loadFactor (16 * 0.75 = 12)
```

HashMap hash spreading and bucket index

4.2 Collision Handling -- LinkedList to TreeNode

```
import java.util.HashMap;
import java.util.Objects;

static final int TREEIFY_THRESHOLD = 8; // chain -> Red-Black tree
static final int UNTREEIFY_THRESHOLD = 6; // tree -> chain on shrink
static final int MIN_TREEIFY_CAPACITY = 64; // table must be >= 64 to treeify

// 1-7 collisions in a bucket: stored as linked list (Node.next chain)
```



```
// 8th collision: if table.length >= 64, treeify to Red-Black tree
// if table.length < 64, resize the table instead

// After treeify: O(log n) per bucket instead of O(n)

// Bad hashCode causes O(n) degradation:
class BadKey {
    int id;

    @Override public int hashCode() { return 42; } // all same bucket!
    @Override public boolean equals(Object o) {
        return o instanceof BadKey b && b.id == id;
    }
}

// Good: spread bits -- use Objects.hash() or record default
record GoodKey(int id, String name) {} // auto-generated hashCode
```

HashMap collision -- linked list to TreeNode treeification

4.3 Resize -- Double Capacity

```
import java.util.HashMap;

// resize() doubles: newCapacity = oldCapacity << 1
// Each entry moves to: same index OR index + oldCapacity
// (determined by checking one extra bit of original hash -- no full re-hash)

// Pre-size to avoid resize:
int expectedEntries = 1000;
int initialCapacity = (int)(expectedEntries / 0.75) + 1; // ~1334
HashMap<String, Integer> scores = new HashMap<>(initialCapacity);

// Java 19+: HashMap.newHashMap(expectedEntries)
// Automatically computes correct initial capacity
```

HashMap resize and pre-sizing

Map Type	Ordering	Null Keys	Thread-Safe	Performance
HashMap	None	1 allowed	No	O(1) avg
LinkedHashMap	Insertion / Access	1 allowed	No	O(1) avg
TreeMap	Sorted (natural/Comparator)	No	No	O(log n)
EnumMap	Enum ordinal	No	No	O(1) fastest
ConcurrentHashMap	None	No	Yes (lock-stripping)	O(1) avg

WARNING: Never mutate a HashMap key after insertion -- changing the key changes its hashCode, making the entry unreachable. Use immutable keys: String, Integer, records.

5 CopyOnWriteArrayList -- Thread-Safe Snapshot Writes

`CopyOnWriteArrayList` (`java.util.concurrent`) is a thread-safe `List` variant for use-cases where reads vastly outnumber writes. On every structural modification, it copies the entire underlying array, makes the change on the new copy, and atomically replaces the volatile reference. Reads need no locking -- they operate on the stable snapshot captured at iterator creation time.

5.1 Snapshot-on-Write Mechanism

```
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.Arrays;
import java.util.Iterator;
import java.util.concurrent.locks.ReentrantLock;

// CopyOnWriteArrayList key fields (OpenJDK simplified)
public class CopyOnWriteArrayList<E> implements List<E> {
    private transient volatile Object[] array; // volatile snapshot
    final transient ReentrantLock lock = new ReentrantLock();

    // add() copies entire array; holds lock during copy
    public boolean add(E e) {
        synchronized (lock) {
            Object[] elements = getArray();
            int len = elements.length;
            Object[] newElements = Arrays.copyOf(elements, len + 1); // O(n)!
            newElements[len] = e;
            setArray(newElements); // volatile write -- atomically visible
            return true;
        }
    }

    // iterator() captures snapshot -- never CME
    public Iterator<E> iterator() {
        return new COWIterator<>(getArray(), 0); // stable old snapshot
    }
}
```

CopyOnWriteArrayList -- snapshot copy on every write

5.2 Ideal Use Case -- Event Listener Registry

```
import java.util.concurrent.CopyOnWriteArrayList;

// Read-heavy: many threads iterate, rare registration changes
CopyOnWriteArrayList<Runnable> listeners = new CopyOnWriteArrayList<>();
```

```
// Multiple threads iterate simultaneously without locks:
for (Runnable l : listeners) {
    l.run(); // safe -- stable snapshot
}

// Concurrent registration (copies array):
listeners.add(() -> System.out.println("listener"));

// Safe to call remove() INSIDE a for-each loop:
for (Runnable l : listeners) {
    listeners.remove(l); // no CME! iterating old snapshot
}
```

CopyOnWriteArrayList as event listener registry

WARNING: CopyOnWriteArrayList is $O(n)$ on every write. Never use it for frequent writes -- it is specifically for read-heavy, write-rarely scenarios.

6 Collections Utility Class

`java.util.Collections` is a final class of static utility methods for `Collection` instances. It provides algorithms (sort, `binarySearch`, `shuffle`), wrappers (`unmodifiableXxx`, `synchronizedXxx`), and factory helpers (`nCopies`, `singleton`, `emptyList`). Knowing these avoids reinventing common algorithms and prevents subtle bugs.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

List<Integer> nums = new ArrayList<>(List.of(5, 3, 8, 1, 9, 2));

// -- Sorting --
Collections.sort(nums); // natural: [1,2,3,5,8,9]
Collections.sort(nums, Comparator.reverseOrder()); // descending

// -- Binary search (list must be sorted) --
Collections.sort(nums);
int idx = Collections.binarySearch(nums, 5); // index, or negative

// -- Shuffle / reverse / rotate --
Collections.shuffle(nums); // Fisher-Yates in-place
Collections.reverse(nums); // reverses in-place
Collections.rotate(nums, 2); // rotate right by 2

// -- Fill / nCopies / frequency / disjoint --
Collections.fill(nums, 0);
List<String> zeros = Collections.nCopies(5, "zero");
int freq = Collections.frequency(nums, 3);
boolean d = Collections.disjoint(nums, List.of(100, 200));

// -- Immutable views (Java 9+ List.of() is preferred) --
List<Integer> unmod = Collections.unmodifiableList(nums);

// -- Synchronized wrappers (legacy -- prefer java.util.concurrent) --
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
synchronized (syncList) { // MUST externally sync during iteration!
    for (int n : syncList) System.out.println(n);
}
```

TIP: Prefer `List.of()`, `Set.of()`, `Map.of()` (Java 9+) over `Collections.unmodifiableXxx` -- they produce truly immutable collections with no mutable backing collection.

7 Fail-Fast vs Fail-Safe Iterators

Java collections fall into two categories regarding concurrent modification during iteration. Fail-fast iterators detect structural modification and immediately throw `ConcurrentModificationException`. Fail-safe iterators operate on a snapshot or use weak consistency, never throwing CME.

7.1 modCount -- The Fail-Fast Mechanism

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

// Every structural modification increments modCount
// (add, remove, clear, addAll -- but NOT set())
List<String> list = new ArrayList<>(List.of("a","b","c"));
Iterator<String> it = list.iterator(); // saves expectedModCount
list.add("d"); // modCount++ (now != expectedModCount)
it.next(); // check fails -> ConcurrentModificationException!

// Safe removal via iterator.remove():
Iterator<String> it2 = list.iterator();
while (it2.hasNext()) {
    if (it2.next().equals("b")) it2.remove(); // adjusts expectedModCount
}

// Safe Java 8+ approach:
list.removeIf(s -> s.equals("b"));

// Safe collect-then-remove:
List<String> toRemove = list.stream().filter("b"::equals).toList();
list.removeAll(toRemove);
```

Fail-fast modCount mechanism and safe patterns

7.2 Fail-Safe Collections Comparison

Collection	Iterator Type	CME?	Sees new elements?	Mechanism
ArrayList, LinkedList, HashMap	Fail-fast	Yes	Yes, then CME	modCount check
CopyOnWriteArrayList	Fail-safe (snapshot)	Never	No (old snapshot)	Iterates copied array
ConcurrentHashMap	Weakly consistent	Never	Maybe (best-effort)	Segment-level iteration
ConcurrentLinkedQueue	Weakly consistent	Never	Maybe	Lock-free traversal

```
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

// ConcurrentHashMap -- weakly consistent, never CME

ConcurrentHashMap<String, Integer> cmap = new ConcurrentHashMap<>();

cmap.put("a", 1); cmap.put("b", 2);

for (Map.Entry<String, Integer> e : cmap.entrySet()) {
    if (e.getValue() == 1) cmap.put("c", 3); // no CME!
    System.out.println(e.getKey() + "=" + e.getValue());
}

// Iterator may or may not see "c" -- weakly consistent
```

ConcurrentHashMap weakly consistent iteration

8 Comparable vs Comparator -- Sorting Deep Dive

Comparable defines the natural ordering of a class (intrinsic, single ordering). Comparator defines an external ordering strategy (extrinsic, multiple orderings possible). Java 8 added factory methods to Comparator enabling fluent, composable multi-field sorting without verbose anonymous classes.

8.1 Comparable -- Natural Ordering

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Product implements Comparable<Product> {

    private final String name;
    private final double price;
    private final int stock;

    public Product(String name, double price, int stock) {
        this.name = name; this.price = price; this.stock = stock;
    }

    @Override
    public int compareTo(Product other) {
        return this.name.compareTo(other.name); // natural: alphabetical
    }

    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getStock() { return stock; }
}

List<Product> products = new ArrayList<>(List.of(
    new Product("Banana", 0.5, 100),
    new Product("Apple", 1.2, 50),
    new Product("Cherry", 3.0, 20)
));

Collections.sort(products); // Apple, Banana, Cherry
```

Comparable natural ordering

8.2 Comparator -- Fluent Chain (Java 8+)

```
import java.util.Comparator;
import java.util.TreeMap;

// Sort by price ascending, then name descending on ties
```

```

Comparator<Product> byPriceThenName =
Comparator.comparingDouble(Product::getPrice)
.thenComparing(Comparator.comparing(Product::getName).reversed());
products.sort(byPriceThenName);

// Null-safe comparator (nulls last)
Comparator<Product> nullSafe =
Comparator.nullsLast(Comparator.comparing(Product::getName));

// Multi-key: stock desc, price asc, name asc
Comparator<Product> complex =
Comparator.comparingInt(Product::getStock).reversed()
.thenComparingDouble(Product::getPrice)
.thenComparing(Product::getName);
products.sort(complex);

// TreeMap with custom ordering
TreeMap<Product, String> byPrice = new TreeMap<>(
Comparator.comparingDouble(Product::getPrice));

```

Comparator.comparing() chain -- multi-field sort

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Location	Inside the class	External / lambda
Orderings per class	1 (natural)	Unlimited
Functional interface?	No	Yes (@FunctionalInterface)
Null handling	Manual	Comparator.nullsFirst/L ast

TIP: Ensure consistency with equals: (a.compareTo(b) == 0) should imply a.equals(b). TreeSet and TreeMap use compareTo for equality -- inconsistency causes silent data loss.

9 Stream-Friendly Collections -- Splitter Characteristics

Splitter (Splittable Iterator) is the engine behind Java 8 streams. It reports characteristics that allow the stream framework to optimize parallel pipelines -- skipping size calculations if SIZED is set, enabling sorted merge if SORTED is set, or safely partitioning work if IMMUTABLE is set.

Characteristic	Bit	Meaning	Collections
SIZED	0x0040	Exact size known	ArrayList, HashSet
ORDERED	0x0010	Elements have encounter order	ArrayList, TreeSet
DISTINCT	0x0001	No duplicates	HashSet, TreeSet
SORTED	0x0004	Elements in sorted order	TreeSet, TreeMap.keySet()
NONNULL	0x0100	No null elements	ConcurrentHashMap
IMMUTABLE	0x0400	Source cannot be modified	List.of(), Set.of()
CONCURRENT	0x1000	Safe concurrent modification	ConcurrentHashMap
SUBSIZED	0x4000	Splits have exact size	ArrayList

```
import java.util.ArrayList;
import java.util.List;
import java.util.Spliterator;
import java.util.TreeSet;

// Inspecting ArrayList Spliterator
List<String> list = new ArrayList<>(List.of("x","y","z"));
Spliterator<String> spl = list.spliterator();
System.out.println(spl.estimateSize()); // 3
System.out.println(
    (spl.characteristics() & Spliterator.SIZED) != 0); // true
System.out.println(
    (spl.characteristics() & Spliterator.SORTED) != 0); // false

// Parallel stream benefits from SIZED + SUBSIZED (even split)
long count = list.parallelStream().filter(s -> !s.isEmpty()).count();

// TreeSet has SORTED + ORDERED + DISTINCT + SIZED
TreeSet<Integer> tset = new TreeSet<>(List.of(5,1,3,2,4));
tset.stream() // sorted 1,2,3,4,5 -- skips sort step in pipeline
    .map(i -> i * 2)
    .forEach(System.out::println);
```

Spliterator characteristics and parallel streams

10 Memory Layout and Performance Summary

Collection	Storage	Access	Insert end	Insert mid	Delete	Search
ArrayList	Object[]	O(1)	O(1) amort	O(n)	O(n)	O(n)
LinkedList	Node chain	O(n)	O(1)	O(1)*	O(1)*	O(n)
ArrayDeque	Circular array	O(1) ends	O(1) amort	N/A	O(1)	O(n)
HashSet	HashMap backing	N/A	O(1) avg	N/A	O(1) avg	O(1) avg
TreeSet	Red-Black Tree	N/A	O(log n)	N/A	O(log n)	O(log n)
HashMap	Node[]+chains	N/A	O(1) avg	N/A	O(1) avg	O(1) avg
TreeMap	Red-Black Tree	N/A	O(log n)	N/A	O(log n)	O(log n)
PriorityQueue	Binary Heap	O(1) peek	O(log n)	N/A	O(log n)	O(n)

* LinkedList O(1) insert/delete at middle requires O(n) traversal to locate the node first.

10.1 Collection Selection Guide

Use Case	Best Choice	Why
Indexed random access	ArrayList	O(1) by index; contiguous array
Frequent add/remove both ends	ArrayDeque	O(1) amortized, no Node overhead
Thread-safe read-many write-rarely	CopyOnWriteArrayList	Lock-free reads on snapshot
Key-value, no order needed	HashMap	O(1) avg, lowest overhead
Key-value, sorted by key	TreeMap	O(log n), floorKey/ceilingKey
Key-value, insertion order	LinkedHashMap	HashMap + doubly-linked access list
Unique values, order irrelevant	HashSet	O(1) contains/add
Unique values, sorted	TreeSet	O(log n), first/last/headSet
Priority scheduling	PriorityQueue	Min-heap, O(log n) poll
Enum keys	EnumMap	Ordinal array, fastest possible Map
Thread-safe concurrent map	ConcurrentHashMap	Lock-striping, never blocks readers

Q Interview Q&A -- 30 Questions

Q1. What is the root interface of the Java Collections Framework?

`java.lang.Iterable<E>` is the root for all collections. `Collection` extends `Iterable`, and `List`, `Set`, `Queue` extend `Collection`. Note: `Map` does NOT extend `Collection`; it is a separate hierarchy rooted at `java.util.Map<K,V>`.

Q2. What is the difference between `ArrayList` and `LinkedList`?

`ArrayList` is backed by `Object[]` giving $O(1)$ random access but $O(n)$ insert/delete in the middle. `LinkedList` is doubly-linked with $O(1)$ add/remove at ends but $O(n)$ get by index. Prefer `ArrayList` for most use-cases due to better cache locality; use `ArrayDeque` for deque/queue operations.

Q3. What is the default initial capacity and growth factor of `ArrayList`?

`DEFAULT_CAPACITY` is 10 (on first element add; no-arg constructor starts with a 0-length array). Growth factor is 1.5x: `newCapacity = oldCapacity + (oldCapacity >> 1)`, implemented in the private `grow()` method using `Arrays.copyOf`.

Q4. How does `HashMap` calculate a bucket index?

`hash = key.hashCode() ^ (key.hashCode() >>> 16)` -- XOR spreads high 16 bits into low bits. `Index = (n-1) & hash`, where `n = table.length` (always power of 2). Bitwise `&` replaces `%` for speed when `n` is power-of-2.

Q5. What are `TREEIFY_THRESHOLD` and `UNTREEIFY_THRESHOLD`?

`TREEIFY_THRESHOLD=8`: when a bucket chain reaches 8 entries AND table length ≥ 64 , convert chain to Red-Black tree ($O(\log n)$ per bucket). `UNTREEIFY_THRESHOLD=6`: after shrink/remove, if tree has < 6 nodes, convert back to linked list.

Q6. What is a fail-fast iterator?

A fail-fast iterator checks `modCount` on every `next()` call. If `modCount` changed since the iterator was created, it throws `ConcurrentModificationException` immediately. `ArrayList`, `LinkedList`, `HashMap` all use fail-fast iterators.

Q7. How do you safely remove elements during iteration?

(1) `iterator.remove()` -- only safe structural removal through iterator. (2) `list.removeIf(predicate)` -- Java 8+, internally uses `iterator.remove()`. (3) Collect to-remove elements then call `list.removeAll(toRemove)` after loop.

Q8. What is `CopyOnWriteArrayList` and when should you use it?

`CopyOnWriteArrayList` creates a full copy of the backing array on every write. Reads are lock-free (operate on captured snapshot). Use for listener/observer registries: read frequently, updated rarely. Never use for frequent writes -- each write is $O(n)$.

Q9. What is the difference between `Comparable` and `Comparator`?

`Comparable` defines natural ordering inside the class via `compareTo(T o)`. One ordering per class. `Comparator` is external (lambda or separate class) via `compare(T o1, T o2)`. Unlimited `Comparators` per class. `Comparator` is a `@FunctionalInterface`; `Comparable` is not.

Q10. What is `Splitter` and how is it different from `Iterator`?

`Splitter` is a splittable iterator for parallel stream pipelines. It adds `trySplit()` (partition work), `estimateSize()` (known size), and `characteristics()` (bitmask: `SIZED`, `ORDERED`, `SORTED`, `DISTINCT` etc.). The stream framework uses these to optimize parallel execution. `Iterator` has none of these capabilities.

Q11. What does `Collections.unmodifiableList()` return?

A view of the original list that throws `UnsupportedOperationException` on mutation. Reads go through to the backing list -- changes to the backing list are reflected in the view. For a truly immutable copy use `List.copyOf()` or `List.of()` (Java 9+).

Q12. Why should `HashMap` keys be immutable?

`HashMap` stores the hash at insertion time. If a key is mutated later, its `hashCode` changes -- the entry is in the wrong bucket and `get/remove` cannot find it (returns null or false). Use `String`, `Integer`, `Long`, `UUID`, or records as keys.

Q13. What is the difference between `HashMap` and `LinkedHashMap`?

`LinkedHashMap` extends `HashMap` and maintains a doubly-linked list of entries for insertion order (default) or access order (new `LinkedHashMap`(cap, lf, true)). Access-order enables a classic LRU cache by overriding `removeEldestEntry()`.

Q14. How does `TreeMap` maintain sorted order?

`TreeMap` is backed by a Red-Black tree (self-balancing BST). Keys are compared via the `Comparator` passed at construction or natural ordering (`Comparable`). All operations are $O(\log n)$. Provides navigation: `floorKey`, `ceilingKey`, `headMap`, `tailMap`, `subMap`.

Q15. What is the difference between `HashMap` and `ConcurrentHashMap`?

`HashMap` is not thread-safe -- concurrent modification causes data corruption or infinite loops. `ConcurrentHashMap` uses per-bucket locking (Java 8+): reads are fully lock-free (volatile), writes lock only the affected bucket (CAS or synchronized on head node). No null keys or values in `ConcurrentHashMap`.

Q16. [ADV] Explain the `HashMap` resize algorithm and why capacity is always a power of 2.

Resize doubles capacity: $\text{newCap} = \text{oldCap} \ll 1$. Power-of-2 allows $(n-1) \& \text{hash}$ to replace $\text{hash} \% n$ (faster). During resize, each entry moves to same index or $\text{index} + \text{oldCap}$ -- determined by checking one additional bit of the original hash. No full re-hash needed.

Q17. [ADV] What is hash spreading in `HashMap` and why is it necessary?

$\text{hash} = h \wedge (h \ggg 16)$ XORs high 16 bits into low bits. Without this, for small tables (16 slots), only the bottom 4 bits of `hashCode()` determine the bucket -- distinct keys with similar low bits collide. Spreading mixes high-entropy upper bits into the lower bits used for indexing.

Q18. [ADV] How does `ConcurrentHashMap` achieve thread safety without full locking?

Java 8+: per-bucket locking. Reads use volatile `Node.val/next` -- fully lock-free. Empty-slot writes use CAS. Non-empty-slot writes synchronize on the bucket head node only. `computeIfAbsent`, `merge` use per-bucket synchronized blocks. Size is tracked with LongAdder-like counter cells to avoid contention.

Q19. [ADV] What counts as structural vs non-structural modification for `modCount`?

Structural: `add`, `remove`, `clear`, `addAll`, `removeAll`, `retainAll`, `ensureCapacity` (when it causes resize) -- all increment `modCount`. Non-structural: `set(index, value)` replaces a value without changing size or structure -- `modCount` NOT incremented. Iterators save `expectedModCount`; `next()` checks equality.

Q20. [ADV] Why does `TreeSet` use `compareTo()` for equality instead of `equals()`?

`TreeSet` uses `TreeMap` internally and relies on `Comparator/Comparable` for equality: if `compareTo()` returns 0, they are the same element. If your `Comparable` is inconsistent with `equals` (`compareTo` returns 0 but `equals` returns false), `TreeSet` treats them as duplicates -- silently dropping elements. The contract: `Comparator` must be consistent

with equals.

Q21. [ADV] What is the time complexity of PriorityQueue operations?

offer/add: $O(\log n)$ -- sift-up. poll/remove: $O(\log n)$ -- sift-down. peek: $O(1)$. contains/remove(Object): $O(n)$ -- linear scan. Heapify constructor: $O(n)$ -- not $O(n \log n)$. Iterator does not return elements in priority order; use repeated poll() for sorted output.

Q22. [ADV] How do Spliterator characteristics affect parallel stream performance?

SIZED+SUBSIZED: splits can be sized evenly without re-measuring. ORDERED: parallel merge must preserve encounter order (overhead). SORTED: pipeline can skip sorting step. DISTINCT: dedup can be skipped. IMMUTABLE: source is safe to read concurrently with no synchronization. Removing ORDERED with unordered() on a parallel stream can dramatically improve throughput.

Q23. [ADV] How do you implement an LRU cache with LinkedHashMap?

new LinkedHashMap<>(cap, 0.75f, true) -- access-order mode. Override removeEldestEntry(eldest) to return size() > maxSize. Each get() and put() moves the entry to the tail (most recently used). Eldest (head) is automatically removed when predicate returns true. Single-threaded $O(1)$ LRU cache without external libraries.

Q24. [ADV] How does EnumSet achieve $O(1)$ performance?

EnumSet is backed by a long bitmask (RegularEnumSet for ≤ 64 constants) or long[] (JumboEnumSet). Each enum constant has an ordinal (0,1,2...) -- membership is a single bit test/set. contains, add, remove, removeAll are all pure bitwise arithmetic. No hashing, no collision resolution, no tree traversal.

Q25. [ADV] What is the difference between List.of() and Collections.unmodifiableList()?

List.of() creates a truly immutable, null-hostile list with no backing mutable collection. Collections.unmodifiableList(list) creates a mutable-backed view -- mutations through the view throw UOE, but the original list reference can still be mutated and the view reflects those changes. List.copyOf() is List.of() for an existing collection.

Q26. [ADV] Explain the Iterator protocol contract.

Iterator contract: hasNext() must not advance the iterator. next() throws NoSuchElementException if no more elements. remove() before next() or called twice in a row throws IllegalStateException. These are specified in Javadoc but not all implementations enforce all three. Robust iterators enforce all of them.

Q27. [ADV] Why is ArrayDeque preferred over Stack and LinkedList for stack/queue operations?

ArrayDeque uses a circular Object[] -- no per-element allocation, better cache locality, lower GC pressure. Stack extends Vector (fully synchronized, legacy). LinkedList allocates ~40 bytes per Node. Java documentation explicitly states ArrayDeque is likely faster than LinkedList as a deque. Exception: ArrayDeque cannot hold null (null is used as circular-buffer sentinel).

Q28. [ADV] What is the modCount check actually checking -- is it guaranteed to catch all concurrent modifications?

modCount is a best-effort check, not a guaranteed concurrent-safety mechanism. The Javadoc explicitly states it is not guaranteed to prevent all concurrency bugs -- it is an aid for debugging. The check can theoretically be fooled if the count wraps around (very unlikely but possible). Never rely on CME as a substitute for proper synchronization.

Q29. [ADV] How does HashMap.get() differ from ConcurrentHashMap.get() at the JVM level?

HashMap.get() uses no volatile reads -- it may see stale data from another thread or even corrupt state. ConcurrentHashMap.get() reads Node.val and Node.next as volatile -- always sees a consistent (though possibly

stale) value. Lock-free ConcurrentHashMap reads never block even during concurrent resize.

Q30. [ADV] Explain Red-Black tree invariants and why they guarantee $O(\log n)$ operations.

Red-Black tree invariants: (1) Every node is RED or BLACK. (2) Root is BLACK. (3) No two consecutive RED nodes (parent+child both RED is forbidden). (4) All paths from root to null leaves have equal BLACK-node count (black-height). These invariants bound tree height to $2 \cdot \log_2(n+1)$, guaranteeing $O(\log n)$ for all operations. Violations after insert/remove are fixed with rotations and recoloring.

! Common Pitfalls and Anti-Patterns

Anti-Pattern	Problem	Fix
Mutable HashMap key	Hash changes; entry unreachable	Use String, Integer, records (immutable)
ConcurrentModificationException in for-each	Structural change during iteration	Use iterator.remove() or removeIf()
Using LinkedList for random access	O(n) per get(index)	Use ArrayList for indexed access
No initial capacity for large inserts	Repeated grow() copies	new ArrayList<>(size) or ensureCapacity()
CopyOnWriteArrayList with frequent writes	O(n) copy on every write	Use ConcurrentHashMap or lock-free structures
Comparator inconsistent with equals in TreeSet	Elements silently merged/lost	Ensure compareTo==0 iff equals==true
synchronizedList without sync on iteration	CME or data corruption	Externally sync on list during iteration
HashMap with bad hashCode (constant)	O(n) or O(log n) per bucket	Use Objects.hash() or records
Calling List.of().add()	UnsupportedOperationException	Wrap in new ArrayList<>(List.of(...))
Using Stack class	Synchronized Vector -- legacy	Use ArrayDeque as stack

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

// WRONG: modifying list in for-each
List<String> list = new ArrayList<>(List.of("a","b","c"));
for (String s : list) {
    if (s.equals("b")) list.remove(s); // ConcurrentModificationException!
}

// FIX: iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().equals("b")) it.remove(); }

// FIX: removeIf (Java 8+)
list.removeIf("b"::equals);
```

```
// WRONG: mutable HashMap key

class MutableKey { int id; }

MutableKey key = new MutableKey(); key.id = 1;

Map<MutableKey, String> map = new HashMap<>();

map.put(key, "value");

key.id = 999; // hashCode changed!

System.out.println(map.get(key)); // null!


// FIX: use record (immutable key)

record SafeKey(int id) {}

Map<SafeKey, String> safe = new HashMap<>();

safe.put(new SafeKey(1), "value");

System.out.println(safe.get(new SafeKey(1))); // "value"
```

Common pitfalls fixed

S Summary -- Key Takeaways

Topic	Key Points
ArrayList	Object[] backing, DEFAULT_CAPACITY=10, 1.5x growth, O(1) random access, O(n) middle ops
LinkedList	Doubly-linked Node, O(1) ends, O(n) by index -- prefer ArrayDeque
HashMap	Node[] table, hash=h^(h>>>16), index=(n-1)&hash, treeify at 8, resize doubles
CopyOnWriteArrayList	Volatile snapshot, O(n) write, lock-free read -- listener registries only
Fail-fast	modCount check in next() -- ArrayList, LinkedList, HashMap throw CME
Fail-safe	ConcurrentHashMap (weak consistency), COWAL (snapshot) -- never CME
Comparable	Natural ordering, one per class, used by sort/TreeSet/TreeMap by default
Comparator	External strategy, unlimited, Java 8+ fluent: comparing().thenComparing().reversed()
Splititerator	SIZED+ORDERED+SORTED+DISTINCT characteristics optimize parallel stream pipelines
Collections util	sort, binarySearch, shuffle, fill, nCopies, frequency, disjoint, unmodifiable

Next: Sub-Chapter 1.7 -- Generics